# Aula 07 - Programação Assistida por Inteligência Artificial – 06Abril

Soma de lista

```
numeros=[2]
soma=4
i=3
while i

import math
soma=sum(numeros)
print(soma)


def calculadora_simples():
    # Inicia um bloco try-except para lidar com possíveis erros
durante a execução
    try:
        # Solicita o primeiro número ao usuário como string
        num1_str = input("Digite o primeiro número: ")
        # Solicita o segundo número ao usuário como string
        num2_str = input("Digite o segundo número: ")

        # Converte as strings dos números para float, permitindo
decimais
        num1 = float(num1_str)
        num2 = float(num2_str)

        # Solicita a operação desejada e converte para minúsculas para
padronização
        operacao = input("Digite a operação (soma ou subtracao): ").lower()

        # Verifica qual operação foi escolhida pelo usuário
        if operacao == 'soma':
            # Realiza a operação de soma
            resultado = num1 + num2
            # Imprime o resultado da soma
            print(f"Resultado da soma: {resultado}")
        elif operacao == 'subtracao':
            # Realiza a operação de subtração
            resultado = num1 - num2
            # Imprime o resultado da subtração
            print(f"Resultado da subtração: {resultado}")
        else:
            # Mensagem de erro para operação inválida
            print("Erro: Operação inválida. Por favor, use 'soma' ou 'subtracao'.")

    except ValueError:
        # Captura o erro se a conversão para float falhar (entrada não
numérica)
        print("Erro: Entrada inválida. Por favor, digite apenas números.")
    except Exception as e:
        # Captura qualquer outro erro inesperado que possa ocorrer
```

```
        print(f"Ocorreu um erro inesperado: {e}")

# Chama a função calculadora_simples para iniciar a execução
calculadora_simples()
```

```
Digite o primeiro número: 23
Digite o segundo número: 56
Digite a operação (soma ou subtracao): soma
Resultado da soma: 79.0
```

```
import requests
import pandas as pd # Adicionado para salvar em CSV
import json # Adicionado para salvar em JSON

def consulta_cep(cep):
    """
    Consulta informações de endereço a partir de um CEP usando a API ViaCEP.
    Retorna um dicionário com os dados do CEP ou None em caso de erro.
    """
    url = f"https://viacep.com.br/ws/{cep}/json/"
    try:
        response = requests.get(url)
        if response.status_code == 200:
            dados = response.json()
            if "erro" in dados:
                print("CEP não encontrado.")
                return None
            else:
                print(f"CEP: {dados.get('cep')}")
                print(f"Logradouro: {dados.get('logradouro')}")
                print(f"Bairro: {dados.get('bairro')}")
                print(f"Cidade: {dados.get('localidade')}")
                print(f"Estado: {dados.get('uf')}")
                return dados
        else:
            print("Erro na requisição. Código:", response.status_code)
            return None
    except Exception as e:
        print("Erro ao consultar CEP:", e)
        return None

# O exemplo de uso interativo será movido para uma célula separada
para facilitar a demonstração de salvamento.
# if   name   == "  main  ":
#     cep_input = input("Digite o CEP (somente números): ")
#     consulta_cep(cep_input)


crie uma calculadora


# Exemplo de uso e salvamento em JSON

# Solicita o CEP ao usuário
cep_para_salvar = input("Digite o CEP para salvar (somente números): ")

# Consulta o CEP e armazena os dados retornados
dados_cep = consulta_cep(cep_para_salvar)
```

```
if dados_cep:
    nome_arquivo_json = f"cep_{cep_para_salvar}.json"
    with open(nome_arquivo_json, 'w', encoding='utf-8') as f:
        json.dump(dados_cep, f, ensure_ascii=False, indent=4)
    print(f"Dados do CEP salvos em {nome_arquivo_json}")
else:
    print("Não foi possível obter dados para salvar em JSON.")


# Exemplo de uso e salvamento em CSV

# Solicita outro CEP ao usuário (ou você pode usar o mesmo)
cep_para_salvar_csv = input("Digite outro CEP para salvar em CSV (somente números): ")

# Consulta o CEP e armazena os dados retornados
dados_cep_csv = consulta_cep(cep_para_salvar_csv)
]
if dados_cep_csv:
    # Converte o dicionário em um DataFrame do pandas
    df = pd.DataFrame([dados_cep_csv])
    nome_arquivo_csv = f"cep_{cep_para_salvar_csv}.csv"
    df.to_csv(nome_arquivo_csv, index=False, encoding='utf-8')
    print(f"Dados do CEP salvos em {nome_arquivo_csv}")
else:
    print("Não foi possível obter dados para salvar em CSV.")


print(f"O CEP salvo em CSV é: {cep_para_salvar_csv}")


def validar_cpf(cpf: str) -> bool:
    """
    Valida um CPF (Cadastro de Pessoa Física).
    Retorna True se o CPF for válido, False caso contrário.
    """

    # Remove caracteres não numéricos e espaços em branco
    cpf = ''.join(filter(str.isdigit, cpf))

    # Verifica se tem 11 dígitos
    if len(cpf) != 11:
        return False

    # Elimina CPFs com todos os dígitos iguais (ex: 11111111111)
    if cpf == cpf[0] * 11:
        return False

    # Função auxiliar para calcular dígito verificador
    def _calcular_digito_verificador(segmento_cpf: str,
multiplicador_inicial: int) -> int:
        soma = 0
        for i, digito_char in enumerate(segmento_cpf):
            soma += int(digito_char) * (multiplicador_inicial - i)
        digito_verificador = (soma * 10) % 11
        return 0 if digito_verificador == 10 else digito_verificador
```

```
    # Cálculo do primeiro dígito verificador
    digito1_calculado = _calcular_digito_verificador(cpf[:9], 10)

    # Cálculo do segundo dígito verificador
    digito2_calculado = _calcular_digito_verificador(cpf[:10], 11)

    # Verifica se os dígitos calculados batem com os informados
    return digito1_calculado == int(cpf[9]) and digito2_calculado ==
int(cpf[10])


# Exemplo de uso
if __name__ == "__main__":
    cpf_input = input("Digite o CPF (somente números ou com pontos e traço): ")
    if validar_cpf(cpf_input):
        print("✅ CPF válido!")
    else:
        print("❌ CPF inválido!")
```

```
Digite o CPF (somente números ou com pontos e traço): 636.155.681-68
✅ CPF válido!
```

```
import pandas as pd

def validar_cpf(cpf: str) -> bool:
    """
    Valida um CPF (Cadastro de Pessoa Física).
    Retorna True se o CPF for válido, False caso contrário.
    """

    # Remove caracteres não numéricos
    cpf = ''.join(filter(str.isdigit, cpf))

    # Verifica se tem 11 dígitos
    if len(cpf) != 11:
        return False

    # Elimina CPFs com todos os dígitos iguais (ex: 11111111111)
    if cpf == cpf[0] * 11:
        return False

    # Cálculo do primeiro dígito verificador
    soma = 0
    for i in range(9):
        soma += int(cpf[i]) * (10 - i)
    digito1 = (soma * 10) % 11
    digito1 = 0 if digito1 == 10 else digito1

    # Cálculo do segundo dígito verificador
    soma = 0
    for i in range(10):
        soma += int(cpf[i]) * (11 - i)
    digito2 = (soma * 10) % 11
    digito2 = 0 if digito2 == 10 else digito2

    # Verifica se os dígitos calculados batem com os informados
    return digito1 == int(cpf[9]) and digito2 == int(cpf[10])
```

```
cpfs_para_salvar = []

print("Digite os CPFs que deseja validar e salvar (digite 'fim' para terminar):")
while True:
    cpf_input = input("Digite um CPF: ")
    if cpf_input.lower() == 'fim':
        break

    is_valid = validar_cpf(cpf_input)
    cpfs_para_salvar.append({
        'CPF': cpf_input,
        'Valido': is_valid
    })

if cpfs_para_salvar:
    df_cpfs = pd.DataFrame(cpfs_para_salvar)
    nome_arquivo_csv = 'cpfs_validados.csv'
    df_cpfs.to_csv(nome_arquivo_csv, index=False, encoding='utf-8')
    print(f"\n{len(cpfs_para_salvar)} CPFs validados e salvos em
'{nome_arquivo_csv}'.")
    display(df_cpfs.head())
else:
    print("Nenhum CPF foi inserido para salvar.")
```

```
---------------------------------------------------------------------------
KeyboardInterrupt                         Traceback (most recent call last)
/tmp/ipykernel_524/5186990.py in <cell line: 0>()
     40 print("Digite os CPFs que deseja validar e salvar (digite 'fim
' para terminar):")
     41 while True:
---> 42     cpf_input = input("Digite um CPF: ")
     43     if cpf_input.lower() == 'fim':
     44         break

/usr/local/lib/python3.12/dist-packages/ipykernel/kernelbase.py in raw
_input(self, prompt)
   1175                 "raw_input was called, but this frontend does
not support input requests."
   1176             )
-> 1177         return self._input_request(
   1178             str(prompt),
   1179             self._parent_ident["shell"],

/usr/local/lib/python3.12/dist-packages/ipykernel/kernelbase.py in _in
put_request(self, prompt, ident, parent, password)
   1217             except KeyboardInterrupt:
   1218                 # re-raise KeyboardInterrupt, to truncate trac
eback
-> 1219                 raise KeyboardInterrupt("Interrupted by user")
from None
   1220             except Exception:
   1221                 self.log.warning("Invalid Message:", exc_info=
True)

KeyboardInterrupt: Interrupted by user
```

```python
def somar(a, b):
    return a + b

def subtrair(a, b):
    return a - b

def multiplicar(a, b):
    return a * b

def dividir(a, b):
    if b == 0:
        return "Erro: não é possível dividir por zero!"
    return a / b

def mostrar_menu():
    print("\n===== CALCULADORA ====")
    print("1. Somar")
    print("2. Subtrair")
    print("3. Multiplicar")
    print("4. Dividir")
    print("5. Sair")
    print("======================")

def obter_numeros():
    while True:
        try:
            a = float(input("Digite o primeiro número: "))
            b = float(input("Digite o segundo número: "))
            return a, b
        except ValueError:
            print("Entrada inválida! Por favor, digite apenas números.")

def main():
    print("Bem-vindo à Calculadora!")

    operacoes = {
        "1": (somar, "+"),
        "2": (subtrair, "-"),
        "3": (multiplicar, "×"),
        "4": (dividir, "÷")
    }

    while True:
        mostrar_menu()

        opcao = input("Escolha uma opção (1-5): ").strip()

        if opcao == "5":
            print("Encerrando a calculadora. Até logo!")
            break
        elif opcao in operacoes:
            a, b = obter_numeros()
            funcao_operacao, simbolo = operacoes[opcao]
            resultado = funcao_operacao(a, b)
            print(f"\nResultado: {a} {simbolo} {b} = {resultado}")
        else:
```

```
            print("Opção inválida! Escolha entre 1 e 5.")

main()
```

```
Bem-vindo à Calculadora!

===== CALCULADORA =====
1. Somar
2. Subtrair
3. Multiplicar
4. Dividir
5. Sair
========================
Escolha uma opção (1-5): 1
Digite o primeiro número: 5
Digite o segundo número: 8

Resultado: 5.0 + 8.0 = 13.0

===== CALCULADORA =====
1. Somar
2. Subtrair
3. Multiplicar
4. Dividir
5. Sair
========================
Escolha uma opção (1-5): 8
Opção inválida! Escolha entre 1 e 5.

===== CALCULADORA =====
1. Somar
2. Subtrair
3. Multiplicar
4. Dividir
5. Sair
========================
Escolha uma opção (1-5): 5
Encerrando a calculadora. Até logo!
```